大数据应用人才培养系列教材

Python语言

刘 鹏　张 燕　○总主编
李肖俊 ○主编　　刘 河　钟 涛 ○副主编

清華大學出版社
TSINGHUA UNIVERSITY PRESS

第八章　类和对象

8.1 理解面向对象

8.2 类的定义与使用

8.3 类的特点

8.4 实验

8.5 小结

8.6 习题

8.1.1 什么是面向对象编程

面向对象编程(Object Oriented Programming)，简称OOP，是一种程序设计思想，是以建立模型体现出来的抽象思维过程和面向对象的方法。模型是用来反映现实世界中事物特征的，是对事物特征和变化规律的抽象，是更普遍、更集中、更深刻地描述客体的特征。OOP把对象作为程序的基本单元，一个对象包含了数据和操作数据的函数。

8.1.2面向对象术语简介

面向对象常用的术语如下：

类：是创建对象的代码段，描述了对象的特征、属性、要实现的功能以及采用的方法等。

属性：描述了对象的静态特征。

方法：描述了对象的动态动作。

对象：对象是类的一个实例，就是模拟真实事件，把数据和代码都集合到一起，即属性、方法的集合。

实例：就是类的实体。

8.1.2面向对象术语简介

实例化：创建类的一个实例的过程。

封装：把对象的属性、方法、事件集中到一个统一的类中，并对调用者屏蔽其中的细节。

继承：一个类共享另一个类的数据结构和方法的机制称为继承。起始类称为基类、超类、父类，而继承类称为派生类、子类。继承类是对被继承类的扩展。

多态：一个同样的函数对于不同的对象可以具有不同的实现，就称为多态。

接口：定义了方法、属性的结构，为其成员提供规约，不提供实现。不能直接从接口创建对象，必须首先创建一个类来实现接口所定义的内容。

8.1.2面向对象术语简介

重载：一个方法可以具有许多不同的接口，但方法的名称是相同的。

事件：事件是由某个外部行为所引发的对象方法。

重写：在派生类中，对基类某个方法的程序代码进行重新编写，使其实现不同的功能，我们把这个过程称为重写。

构造函数：是创建对象所调用的特殊方法。

析构函数：是释放对象时所调用的特殊方法。

第八章　类和对象

8.1 理解面向对象

8.2 类的定义与使用

8.3 类的特点

8.4 实验

8.5 小结

8.6 习题

8.2.1 类的定义

类就是对象的属性和方法的封装，静态的特征称为属性，动态的动作称为方法。

类通常的语法格式如下：

```
class ClassName:
    #属性
    [属性定义体]
    #方法
    [方法定义体]
```

8.2.1 类的定义

类的定义以关键字class开始，类名必须以大写字母开头，类名后面紧跟冒号“:”。

类的定义示例如下：

```
class Person: # Python的类名约定以大写字母开头
  #----类的一个示例----
  # 属性
  skincolor = "yellow"
  high = 168
```

8.2.1 类的定义

```
weight = 65
 #方法
 def goroad(self):
        print("人走路动作的测试……")
 def sleep(self):
        print("睡觉，晚安！")
```

从上例可以看出，属性就是变量，静态的特征，方法就是一个个的函数，通过这些函数来描述动作行为。

8.2.1 类的定义

在定义类属性的时候一般用名词，定义类方法的时候一般用动词，类名约定以大写字母开头，函数约定以小写字母开头。

8.2.2 类的使用

类定义好之后，就可将类实例化为对象。类实例化对象的语法格式如下：

```
对象名 = 类名（）
```

实例化对象的操作符是：等号“=”，在类实例化对象的时候，类名后面要添加一个括号“()”。

类实例化示例：

```
>>> p = Person()
```

上例将类Person实例化为对象p。

8.2.3 类的构造方法及专有方法

类的构造方法是：__init__(self)。

只要实例化一个对象的时候，这个方法就会在对象被创建的时候自动调用。实例化对象的时候是可以传入参数的，这些参数会自动传入__init__(self,param1,param2,…)方法中,我们可以通过重写这个方法来自定义对象的初始化操作。如下例所示代码：

```
>>> class Bear:
        def __init__(self,name):
            self.name = name
        def kill(self):
```

8.2.3 类的构造方法及专有方法

```
print("%s,是保护动物，不能杀..."% self.name)
```

运行结果如下：

```
>>> a = Bear('狗熊')
>>> a.kill()
狗熊,是保护动物，不能杀...
```

在上例中，我们重写了__init__(self)方法，如果没有重写，它的默认调用就是__init__(self),没有任何参数或只有一个self参数，所以在实例化的时候，参数是空的。在例子中，我们给了它一个参数name，就成了__init__(self,name),在实例化的时候就可以传参数了，因为第一个参数self

8.2.3 类的构造方法及专有方法

是默认的，就把“狗熊”传给“name”，运行程序后得到了我们期望的结果，这样使用起来就非常方便。

另外，我们还可以把传入的参数设置为默认参数，我们在实例化的时候不传入参数系统也不会报错，如下例所示代码：

```
>>> class Bear:
        def __init__(self,name = "默认的熊"):
            self.name = name
        def kill(self):
            print("%s,是保护动物，不能杀..."% self.name)
```

8.2.3 类的构造方法及专有方法

代码运行结果如下：

```
>>> b = Bear()
>>> b.kill()
默认的熊,是保护动物，不能杀...
>>> c = Bear('替代熊')
>>> c.kill()
替代熊,是保护动物，不能杀...
```

在上例中，我们把构造函数的参数name设置为默认值：“默认的熊”

8.2.3 类的构造方法及专有方法

对象实例化的时候没有传值给参数name，程序运行正确，输出了期望的结果：“默认的熊,是保护动物，不能杀…”，当在对象实例化的时候给参数name传值为：“替代熊”，当对象c调用方法kill()时，输出了正确的结果：“替代熊,是保护动物，不能杀...”。

8.2.4 类的访问权限

大家都知道，在C++/java中，是通过关键字public、private来表明访问的权限是公有、私有的。然而在Python中，默认情况下对象的属性和方法都是公开的，公有的，通过点(.)操作符来访问，如下例所示代码：

```
>>> class Company:
        name = "云创科技"
```

运行结果如下：

```
>>> c = Company()
>>> c.name
'云创科技'
```

8.2.4 类的访问权限

在上例中，我们直接通过点(.)来访问了类Company的变量name，运行得到了结果：云创科技。

为了实现类似于私有变量的特征，Python内部采用了name mangling的技术（名字重整或名字改变），在变量名或函数名前加上两个下划线（“__”），这个函数或变量就变为私有了。

为了访问类中的私有变量，有一个折衷的处理办法，如下例所示代码：

```
>>> class Company:
        __name = "云创科技"
        def getname(self):
```

8.2.4 类的访问权限

```
return self.__name
```

运行结果如下：

```
>>> c = Company()
>>> c.getname()
'云创科技'
```

在上例中，我们在类Company内部重新定义一个方法getname(self)，在程序外部通过访问对象的getname()方法来访问了类Company中的私有变量：__name。

8.2.4 类的访问权限

```
return self.__name
```

运行结果如下：

```
>>> c = Company()
>>> c.getname()
'云创科技'
```

在上例中，我们在类Company内部重新定义一个方法getname(self)，在程序外部通过访问对象的getname()方法来访问了类Company中的私有变量：__name。

8.2.4 类的访问权限

实际上，Python把双下划线“__”开头的变量名，改为了：单下划线“_”类名+双下划线“__”变量名，即【_类名__变量名】，因此我们可以通过以下的访问方式来访问类的私有变量，如下例所示代码：

```
>>> class Company:
        __name = "云创科技"
        def getname(self):
            return self.__name
```

运行结果如下：

```
>>> c = Company()
```

8.2.4 类的访问权限

```
>>> c.getname()
'云创科技'
>>> c._Company__name
'云创科技'
```

在上例中，我们访问了对象c的_Company__name，运行得到了期望的结果。

从上述可见，就目前而言，Python的私有机制是伪私有，Python的类是没有权限控制的，变量是可以被外部调用的。

8.2.5 获取对象信息

类实例化对象之后，对象就可以调用类的属性和方法，语法格式如下：

对象名 . 属性名

对象名 . 方法名

对象调用类的属性或方法的操作符是：点“.”。

对象调用属性或方法的示例：

```
>>> p.goroad()
```

人走路动作的测试……

```
>>> p.skincolor
```

8.2.5 获取对象信息

```
'yellow'

>>> p.sleep()

睡觉，晚安！

>>> p.high

168

>>> p.weight

65
```

在上例中，对象p分别调用了类Person的方法goroad、属性skincolor、方法sleep、属性high、属性weight。

第八章　类和对象

8.1 理解面向对象

8.2 类的定义与使用

8.3 类的特点

8.4 实验

8.5 小结

8.6 习题

8.3.1 封装

从形式上看，对象封装了属性就是变量，而方法和函数是独立性很强的模块，封装就是一种信息掩蔽技术，使数据更加安全。

例如，列表(list)是Python的一个序列对象，我们要对列表进行调整，如下所示代码：

```
>>> list1 = ['K','J','L','Q','M']

>>> list1.sort()

>>> list1

['J', 'K', 'L', 'M', 'Q']
```

在上例中，我们调用了排序函数sort()对无序的列表进行了正序排序。

8.3.1 封装

由此可见，Python的列表就是对象，它提供了若干种方法来供我们根据需求调整整个列表，但是我们不知道列表对象里面的方法是如何实现的，也不知道列表对象里面有哪些变量，这就是封装。它封装起来，只给我们需要的方法的名字，然后我们调用这个名字，知道它可以实现就可以了，然而它没有具体告诉我们是怎么实现的。

8.3.2 多态

不同对象对同一方法响应不同的行动就是多态。

如下所示代码：

```
>>> class Test_X:
        def func(self):
            print("测试X...")
>>> class Test_Y:
        def func(self):
            print("测试Y...")
```

8.3.2 多态

运行结果如下：

```
>>> x = Test_X()
>>> y = Test_Y()
>>> x.func()
测试X...
>>> y.func()
测试Y...
```

在上例中，我们分别定义了2个类： Test_X， Test_Y，每个类里面都定义了同名函数func，函数func分别实现的功能是：输出字符串“测试X...”

8.3.2 多态

“测试Y...”，类Test_X()的实例对象为：x，类Test_Y()的实例对象为：y，对象x和对象y分别调用了同名函数func()，分别运行得到了不同的结果：测试X…和测试Y…。由此可见，不同对象调用了同名方法(函数),实现的结果不一样，这就是多态。

注意：self相当于C++的this指针。由同一个类可以生成无数个对象，这些对象都来源于同一个类的属性和方法，当一个对象的方法被调用的时候，对象会将自身作为第一个参数传给self参数，接收到self参数的时候，Python就知道是哪一个对象在调用方法了。

8.3.3 继承

继承是子类自动共享父类的数据和方法的机制。

语法格式如下：

```
Class ClassName(BaseClassName):
        ……
```

ClassName：是子类的名称，第一个字母必须大写。

BaseClassName：是父类的名称。

被继承的类我们称为基类、父类或超类，而继承者我们称为子类。子类可以继承父类的任何属性和方法。

我们以列表对象为例，如下所示程序代码：

8.3.3 继承

```
>>> class Test_list(list):
        pass
```

运行结果如下:

```
>>> list1 = Test_list()
>>> list1.append('O')
>>> list1
['O']
>>> list1.append('MT')
>>> list1
```

8.3.3 继承

```
['O', 'MT']

>>> list1.sort()

>>> list1

['MT', 'O']
```

在上例中，我们定义了一个类Test_list()，它继承list，然后将它赋值给变量list1，list1分别调用了append()、sort()方法，得到了我们所期望的结果。从中可以看出，我们定义了类Test_list()，它继承了属于list的属性和方法，所以，我们就可以在对象Test_list里面使用了，这就是继承。

8.3.3 继承

在使用类继承机制的时候，有几点需要注意：

(1)如果子类中定义与父类同名的方法或属性，则会自动覆盖父类对应的方法或属性。

(2)子类重写了父类的方法就会把父类的同名方法覆盖，如果被重写的子类同名的方法里面没有引入父类同名的方法，实例化对象要调用父类的同名方法的时候，程序就会报错。

要解决上述问题，就要在子类里面重写父类同名方法的时候，先引入父类的同名方法。要实现这个继承的目的，有2种技术可采用：一是调用未绑定的父类方法，二是使用super函数。

我们先看看“调用未绑定的父类方法”的技术，它的语法格式如下：

8.3.3 继承

paraname.func(self)

语法各项解释如下：

Paraname——父类的名称；

.——点操作符；

func——子类要重写的父类的同名方法名称；

self——子类的实例对象，注意这里不是父类的实例对象。

接下来，我们看看“使用super函数”的技术，super函数可以自动找到基类的方法和传入self参数，它的语法格式如下：

super().func([parameter])

8.3.3 继承

语法各项解释如下：

super()——super函数；

.——点操作符

func——子类要重写的父类的同名方法名称；

parameter——可选参数，如果参数是self可以省略。

使用super函数的方便之处在于不用写任何基类的名称，直接写重写的方法就可以了，这样Python会自动到基类去寻找，尤其是在多重继承中，或者子类有多个祖先类的时候，super函数会自动到多种层级关系里面去寻找同名的方法。使用super函数带来一个好处，如果以后我们要更改基类，直接修改括号“（）”里面的基类名称就可以了，不用再修改重写的同名方法里面的内容。

8.3.4 多重继承

可以同时继承多个父类的属性和方法，称为多重继承。语法格式如下：

Class ClassName(Base1, Base2, Base3):

……

ClassName ——子类的名字；

Base1, Base2, Base3——基类1的名字，基类2的名字，基类3的名字；有多少个基类，名字依次写入即可。

虽然多重继承的机制可以让子类继承多个基类的属性和方法使用起来很方便，但很容易导致代码混乱，有时候会引起不可预见的Bug，对程序而言不可预见的Bug几乎就是致命的。因此，当我们不确定必须要使用“多重继承”语法的时候，尽量避免使用它。

第八章　类和对象

8.1 理解面向对象

8.2 类的定义与使用

8.3 类的特点

8.4 实验

8.5 小结

8.6 习题

8.4 实验

8.4.1 声明类

8.4.2 类的继承和多态

8.4.3 复制对象

第八章　类和对象

8.1 理解面向对象

8.2 类的定义与使用

8.3 类的特点

8.4 实验

8.5 小结

8.6 习题

8.5 小结

我们发现想要设计一门出色的语言，就要从现实世界里面去寻找，学习，并归纳抽象出真理包含到其中。

继承可以把父类的所有功能都直接拿过来，这样就不必重零做起，子类只需要新增自己特有的方法，也可以把父类不适合的方法覆盖重写。

继承可以一级一级地继承下来。而任何类，最终都可以追溯到根类。

有了继承，才能有多态。

第八章　类和对象

8.1　理解面向对象

8.2　类的定义与使用

8.3　类的特点

8.4　实验

8.5　小结

8.6　习题

习题：

1. 面向对象程序设计的特点分别为？
2. 假设c为类C的对象且包含一个私有数据成员“__name”，那么在类的外部通过对象c直接将其私有数据成员“__name”的值设置为kate的语句可以写作什么？

感谢聆听